RISK_ANALYSIS

Conflict-Resolution + Danger-Zone Analysis — clickup_sync

Field	Value
Revision	1
Last modified	2026-06-09T00:00:00Z
Anti-bluff	§11.4.6 — the "which side is newer" rule is stated precisely; assumptions flagged `UNCONFIRMED:`.

The two sides

- **Side L (Local):** our SQLite single-source-of-truth (§11.4.93/ §11.4.95 workable-items DB) — the ATMOSphere Issues/Fixed items.
- **Side C (ClickUp):** the List `901818394542` board.

Chosen conflict-resolution mechanism (precise, no guessing)

Last-Write-Wins keyed on authoritative edit timestamps, with echo-suppression and a reconciling full-sweep back-stop. Per synced item we persist in SQLite:

column	meaning
`clickup_task_id`	the C-side id (NULL until first push)
`local_updated`	our last local mutation time (ms, our clock)
`clickup_date_updated`	the **C-side `date_updated`** we last saw (ms, ClickUp clock) — the authoritative C timestamp (API_REFERENCE.md §h)
`last_synced_rev`	

column	meaning
	monotonic sync revision we stamped on BOTH sides at the last successful sync
`source_of_truth`	which side last won (`L` / `C`) — audit only
`pending_outbound`	a mutation staged but not yet ack'd by C (idempotency)

"Which side is newer" determination (exact):

1. On a C-side change (webhook `taskUpdated`/`taskStatusUpdated`/… OR sweep-diff), read the task's `date_updated` (C clock).
2. If `clickup_date_updated_new == clickup_date_updated_stored` **AND** the webhook `history_items[].user` is OUR sync bot identity → it's an **echo of our own write** → suppress, just advance `last_synced_rev`. (Echo detection: match `history_items[].user` to the sync-bot user id, AND/OR match a `last_synced_rev` marker we stamped into a ClickUp custom field/tag. We do NOT compare cross-clock timestamps for echo detection — see clock-skew below.)
3. Else a **real** conflict test: compare the change that happened on each side **since `last_synced_rev`**. If only one side changed → that side wins (apply it to the other). If BOTH changed since `last_synced_rev` → **LWW: the side whose authoritative edit is newer wins**, where "newer" is decided **per-clock, not cross-clock**: we record at each sync the pair `(local_updated, clickup_date_updated)`; a side "changed since sync" iff its own current timestamp > its own stored timestamp. If both changed, the winner is the side with the **later wall-clock edit using each side's own monotonic-within-side timestamp**, tie-broken by **C wins** (ClickUp is the human-facing board; a human edit should not be silently overwritten by an automated local change). The losing side's prior value is written to an `conflict_log` table (never silently dropped — §9 data safety).
4. **Who edited last** for attribution (§11.4.104) comes ONLY from webhook `history_items[].user` — the task GET does NOT expose a last-editor (API_REFERENCE.md §h). The local side knows its own editor.

> Cross-clock comparison is AVOIDED for correctness: ClickUp `date_updated` (ClickUp's clock) and `local_updated` (our host clock) are NOT directly comparable. We compare each side against its OWN stored baseline ("did THIS side change since last sync?"), then resolve a both-changed conflict by the human-priority tie-break (C wins) — never by subtracting two different clocks. This eliminates the clock-skew failure mode by construction.

Danger zones → mitigations (each rock-solid)

#	Danger zone	Mitigation
1	**Internet down mid-sync**	All outbound mutations staged in `pending_outbound` with an **idemp** mutations (idempotent → no dup). Inbound: webhooks missed du backstop (#3). Nothing committed to SQLite until the C-side ack mit).
2	**429 rate-limit storm**	Token bucket sized to the **100 req/min** floor (API_REFERENCE. epoch (NOT `Retry-After` — `UNCONFIRMED`: it exists), then exponenti Coalesce bursts; batch reads via `Get Filtered Team Tasks` (100/pa
3	**Webhook missed / not delivered**	Webhooks are at-least-once & possibly-missed (API_REFERENCE `team/{team}/task?list_ids[]=901818394542&date_updated_gt=<lastSeenMs>&order_by=up` is the authoritative backstop — webhooks are an optimization, th
4	**Webhook duplicated / redelivered**	Dedupe by `history_items[].id` (and `webhook_id+event+task_id+da` idempotent.
5	**Concurrent multi-user edit**	LWW with the both-changed conflict path (above); loser value pr users → still one C-side `date_updated`; attribution via `history_items`
6	**Our-echo vs real-edit ambiguity**	Two independent signals (use both): (a) `history_items[].user ==` `last_synced_rev` stamped into a dedicated ClickUp custom field (c the incoming task's marker == the rev we just wrote, it's our ecl
7	**Partial write** (some fields applied, then failure)	Outbound mutation is staged whole; SQLite commit is a single tr dpoints possible, and on failure we re-derive desired state and re reflecting a state C never accepted — commit SQLite only after C
8	**Clock skew**	Eliminated by construction: never compare ClickUp clock vs hos side compared to its OWN baseline.
9	**Deleted-then-re-created**	ClickUp delete = trash, NO trash API (API_REFERENCE.md §k). item `Deleted/Obsolete` (§11.4.90) and clear `clickup_task_id`. A later new item; we link via our stable item key (e.g. an external-ref cu binds to the original local item if the marker matches.
10	**ID remap** (folder/list/board	We key on the **immutable numeric ids** (folder 901814301358 / URL slugs. A `taskMoved` webhook updates the task's list_id; the sw our list is detected as a disappearance and handled like a delete-

#	Danger zone	Mitigation
	moved or renamed)	

Backstop summary (the "rock-solid" layer)

1. **Idempotency keys** on every outbound mutation.
2. **Per-item rev/timestamp triplet** (`local_updated`, `clickup_date_updated`, `last_synced_rev`) in SQLite.
3. **Sync-marker** stamped on the ClickUp side (custom field or tag) for echo-suppression + recreate re-binding.
4. **Transactional staging + atomic commit** — SQLite reflects only C-ack'd state.
5. **Exponential backoff** driven by `X-RateLimit-Reset`.
6. **Webhook `X-Signature` HMAC-SHA256-hex verify** + dedupe by `history_items[].id`.
7. **Reconciling full-sweep cron** (`date_updated_gt`) as the never-miss backstop — webhooks optimize latency, the sweep guarantees convergence.

UNCONFIRMED (Phase-1 verify before relying on)

- `Retry-After` on 429 (use `X-RateLimit-Reset`).
- Webhook retry count / auto-disable threshold (`…/docs/webhookhealth`).
- Public API ability to create the sync-marker custom field (else operator-seeds in UI; or use a tag marker).
- Tag auto-create on `POST /task/{id}/tag/{name}`.
- `date_updated` exact ms-string format (no live task on the empty list yet).